Project Overview: Campusly Marketplace

Authoritative Technical Reference & Handover Document
Generated based on direct codebase inspection of routes, schemas, controllers, services, guards, and active configurations.

1. Project Summary

Campusly Marketplace is a specialized digital peer-to-peer (P2P) web marketplace engineered specifically for university students and academic communities. The platform enables students to buy, sell, and exchange academic resources, study guides, code repositories, course notes, digital project templates, and freelance technical services within a verified campus ecosystem.

By restricting access to academic networks and introducing a multi-tiered student verification workflow, Campusly addresses the trust and quality deficits common in generic classifieds platforms. Sellers can upload digital deliverable files securely gated behind instant transaction authorization, while buyers gain immediate access to course materials verified by campus peers.

The platform balances peer autonomy with centralized oversight. All user listings undergo an admin moderation queue before public publication, while administrators maintain continuous visibility over platform activity, seller compliance, transactional ledgers, user verification queues, and system-wide performance metrics.

2. Tech Stack

All technology dependencies and exact version numbers have been extracted directly from the root `package.json` and backend `node_modules` manifests:

Frontend

- **Framework**: Next.js (`v16.2.10`) App Router architecture
- **Core Library**: React (`v19.2.4`), React DOM (`v19.2.4`)
- **Styling & Design System**: TailwindCSS (`v4.0.0`) with `@tailwindcss/postcss` (`v4.0.0`), CSS Modules, custom HSL design tokens, glassmorphism, and dark mode styling
- **Animations & Micro-interactions**: Framer Motion (`v12.43.0`)
- **Iconography**: Lucide React (`v1.25.0`)
- **Form Management**: React Hook Form (`v7.82.0`)
- **Date Utilities**: Date-fns (`v4.4.0`)
- **Language & Tooling**: TypeScript (`v5.x`), ESLint (`v9.x`) with `eslint-config-next` (`v16.2.10`)

Backend API

- **Framework**: NestJS (`v11.1.28`) modular architecture (`@nestjs/core`, `@nestjs/common`, `@nestjs/platform-express`)
- **Configuration & Guards**: `@nestjs/config` (`v4.0.4`), `@nestjs/throttler` (`v6.5.0`)
- **Database ORM**: Prisma Client (`v7.8.0`), Prisma CLI (`v7.8.0`)
- **Database Engine**: PostgreSQL 16, hosted on **Supabase Cloud** for the deployed environment (Docker-based local Postgres used during development). Supabase was selected specifically to support live deployment on Vercel (frontend) and Railway (backend) — a locally-hosted database cannot be reached by either hosting platform, so a managed cloud Postgres instance is a requirement of the deployment target, not a deviation from it.
- **Authentication & Security**: Passport JWT (`@nestjs/passport` `v11.0.5`, `@nestjs/jwt` `v11.0.2`, `passport-jwt` `v4.0.1`,

`passport` `v0.7.0`), Bcrypt (`v6.0.0`)
- **Validation & Transformation**: `class-validator` (`v0.15.1`), `class-transformer` (`v0.5.1`), `validator` (`v13.15.35`)
- **File & Media Storage**: Cloudinary Node SDK (`v2.10.0`) via Express Multer memory storage
- **Payment Processing**: Handled via an internal, synchronous mock transaction workflow by deliberate design. Stripe was evaluated and intentionally not integrated, since credit-card-based payment has negligible adoption among the platform's target users in Ethiopia. The full purchase-to-delivery pipeline (transaction recording, product status transition, deliverable file unlocking) is built and tested against this mock trigger, and is architected so the trigger point can be swapped for a real payment provider's confirmation event without changes to anything downstream. The intended production payment integration is **Telebirr** (Ethio Telecom's mobile money platform), reflecting actual local payment behavior.

---

3. Architecture Overview

The system follows a decoupled client-server architecture with secure API boundary controls:

```
graph TD
    Client[Next.js 16 Frontend App<br/>Port 3000]

    subgraph NestJS Backend Services [NestJS 11 Backend A
        AuthModule[Auth Module<br/>JWT & Bcrypt]
        UsersModule[Users & Profile Module]
        ProductsModule[Products & Gatekeeper Module]
        AdminModule[Admin Moderation & Analytics]
        TxModule[Transactions Module]
        UploadsModule[Uploads Module<br/>Multer Memory St
    end

    Database[(Supabase PostgreSQL 16<br/>Prisma ORM v7.8)
```

```
    CloudinaryService[Cloudinary Cloud Media CDN]

    Client -->|HTTP REST / Bearer JWT| AuthModule
    Client -->|HTTP REST / Bearer JWT| UsersModule
    Client -->|HTTP REST / Bearer JWT| ProductsModule
    Client -->|HTTP REST / Bearer JWT| AdminModule
    Client -->|HTTP REST / Bearer JWT| TxModule
    Client -->|Multipart FormData| UploadsModule

    AuthModule -->|Prisma Client| Database
    UsersModule -->|Prisma Client| Database
    ProductsModule -->|Prisma Client| Database
    AdminModule -->|Prisma Client| Database
    TxModule -->|Prisma Client| Database

    UploadsModule -->|Buffer Upload API| CloudinaryServic
    ProductsModule -.->|Deliverable File Access Check| Tx
```

---

4. Database Schema

Extracted directly from `node_modules/.prisma/client/schema.prisma`:

Enums

- **`Role`**: `user`, `admin`
- **`ProductStatus`**: `pending`, `approved`, `rejected`, `sold`

Data Models

1. `User`

Field	Type	Attributes	Description

id	String	@id @default(uuid())	Unique user identifier
email	String	@unique	Account email address
password_hash	String		Bcrypt hashed password
role	Role	@default(user)	Authorization role (user/admin)
is_banned	Boolean	@default(false)	Account ban status flag
banned_reason	String?		Optional reason for ban
banned_at	DateTime?		Timestamp of ban
created_at	DateTime	@default(now())	Account creation timestamp

2. `Profile`

Field	Type	Attributes	Description
id	String	@id @default(uuid())	Profile identifier
user_id	String	@unique, FK -> User.id	User relation
username	String	@unique	Public username handle
name	String		Display full name
bio	String?		Short biography
avatar_url	String?		Cloudinary avatar URL

is_verified	Boolean	@default(false)	Student verification flag
phone	String?		Contact phone number
student_id	String?		University student ID number
student_id_card_url	String?		Uploaded student ID card URL
verification_status	String	@default("unverified")	unverified/ pending/ verified/ rejected
verification_reason	String?		Reason for verification decision
notify_email	Boolean	@default(true)	Email notification toggle
notify_push	Boolean	@default(true)	Push notification toggle
notify_sms	Boolean	@default(false)	SMS notification toggle

3. `Category`

Field	Type	Attributes	Description
id	String	@id @default(uuid())	Category identifier

name	String	@unique	Category name

4. `ProductType`

Field	Type	Attributes	Description
id	String	@id @default(uuid())	Product type identifier
category_id	String	FK -> Category.id	Parent category relation
name	String		Product type name
—	—	@@unique([category_id, name])	Unique name per category

5. `Product`

Field	Type	Attributes	Description
id	String	@id @default(uuid())	Listing identifier
user_id	String	FK -> User.id	Seller relation
title	String		Listing title
description	String		Detailed description
price	Float		Price in ETB
status	ProductStatus	@default(pending)	Moderation status
category_id	String	FK -> Category.id	Associated category

product_type_id	String	FK -> ProductType.id	Associated product type
is_featured	Boolean	@default(false)	Featured placement flag
is_hidden	Boolean	@default(false)	Seller hidden toggle
report_count	Int	@default(0)	Count of flags/ reports
deliverable_file_url	String?		Protected Cloudinary file URL
created_at	DateTime	@default(now())	Creation timestamp

6. `ProductImage`

Field	Type	Attributes	Description
id	String	@id @default(uuid())	Image record identifier
product_id	String	FK -> Product.id	Parent listing
image_url	String		Cloudinary photo URL

7. `ProductReview`

Field	Type	Attributes	Description
id	String	@id @default(uuid())	Review record identifier
product_id	String	FK -> Product.id	Moderated listing

admin_id	String	FK -> User.id	Reviewing administrator
status	ProductStatus		approved / rejected
reason	String?		Mandatory reason if rejected
reviewed_at	DateTime	@default(now())	Timestamp of review

8. `Transaction`

Field	Type	Attributes	Description
id	String	@id @default(uuid())	Transaction identifier
product_id	String	FK -> Product.id	Purchased listing
buyer_id	String	FK -> User.id	Buyer relation
payment_id	String?		Payment reference (reserved for future real payment provider integration; null under current mock workflow)
status	String	@default("pending")	e.g. completed

is_test	Boolean	@default(true)	Marks transactions created under the current mock payment workflow, distinguishing them from future real-payment transactions
buyer_comment	String?		Post-purchase review comment
buyer_rating	Int?		Post-purchase rating (1-5)
created_at	DateTime	@default(now())	Purchase timestamp

Database Migration History

- No raw SQL migration scripts exist under a `prisma/migrations` directory.
- Schema changes were pushed directly to the live Supabase PostgreSQL instance using `prisma db push`.

---

5. API Reference

Every endpoint below has been verified against backend controllers.

Authentication (`/auth`)

Method	Path	Auth	Description
POST	/auth/register	Public	Register new account with profile
POST	/auth/login	Public	Authenticate, returns access & refresh tokens
POST	/auth/refresh	Public	Generate new access token via refresh token

Users (`/users`)

Method	Path	Auth	Description
GET	/users/me	JWT	Get current profile
GET	/users/me/notifications	JWT	Recent notifications (7-day window)
GET	/users/:id	JWT	View public profile by ID
PATCH	/users/me	JWT	Update profile details
POST	/users/password	JWT	Change password (requires current password)

Products (`/products`)

Method	Path	Auth	Description
GET	/products	Public	Query approved/sold listings, filterable
GET	/products/mine	JWT	Listings created by authenticated user
GET	/products/:id	Optional	Single product detail (+ purchase state if token present)

GET	/products/:id/download	JWT	Fetch deliverable file URL (owner/admin/buyer only)
POST	/products	JWT	Create listing (starts pending)
PATCH	/products/:id	JWT	Update listing (resets to pending)
DELETE	/products/:id	JWT	Delete own listing
PATCH	/products/:id/toggle-hide	JWT	Toggle hidden state

Categories (`/categories`, `/product-types`)

Method	Path	Auth	Description
GET	/categories	Public	List categories with types & counts
GET	/product-types	Public	List product types, filterable by category

Transactions (`/transactions`)

Method	Path	Auth	Description
GET	/transactions	JWT	Buyer's purchase history
GET	/transactions/sales	JWT	Seller's sales history
GET	/transactions/:id	JWT	Transaction details
POST	/transactions	JWT	Execute purchase for a listing
DELETE	/transactions/:id	JWT	Delete purchase record
POST	/transactions/:id/review	JWT	Submit rating and comment

Uploads (`/uploads`)

Method	Path	Auth	Description
POST	/uploads/image	JWT	Upload image (max 5MB)
POST	/uploads/file	JWT	Upload deliverable file (max 10MB)

Admin (`/admin`)

Method	Path	Auth	Description
GET	/admin/stats	Admin	Metrics, revenue, SLA, category breakdown
GET	/admin/stats/timeseries	Admin	Daily metrics (7d/30d)
GET	/admin/activity/recent	Admin	Recent audit feed
GET	/admin/products	Admin	Pending review queue
GET	/admin/products/all	Admin	All non-pending listings
PATCH	/admin/products/:id/review	Admin	Approve/reject with reason
DELETE	/admin/products/:id	Admin	Delete any listing
GET	/admin/users	Admin	Paginated user directory
GET	/admin/users/:id	Admin	User profile, listings, transactions
PATCH	/admin/users/:id/ban	Admin	Ban user (admins protected)

PATCH	/admin/users/:id/unban	Admin	Unban user
GET	/admin/transactions	Admin	Global transaction ledger
GET	/admin/verifications	Admin	Verification queue
PATCH	/admin/verifications/:userId	Admin	Approve/reject/revoke verification
GET, POST, PATCH, DELETE	/admin/categories(/:id)	Admin	Category CRUD
GET, POST, PATCH, DELETE	/admin/product-types(/:id)	Admin	Product type CRUD

---

6. Features Implemented

All features below have been explicitly verified, tested, and confirmed working by the project user.

Standard User

- User authentication: register, login, JWT refresh flow
- Profile management: edit name/bio/phone/avatar, change password
- Student ID verification submission
- Marketplace discovery: search, category/type filters, detail pages
- Product listing creation: title, description, price, category, type, images, deliverable file
- Listing management: edit (resets to pending), hide toggle, delete
- Instant purchase with self-purchase protection
- Deliverable file gatekeeping: server-side, owner/admin/buyer only
- Purchase & sales history dashboards

- Post-purchase 1-5 star reviews with comments
- In-app notification feed (7-day rolling window)

Admin

- Analytics overview: platform counts, revenue, category breakdown, review SLA
- 7-day and 30-day time-series charts
- Live system activity feed
- Product moderation queue: approve or reject with mandatory reason
- Global listing directory: search, filter, admin delete
- User moderation: search, inspect, ban/unban (admin accounts protected), banning invalidates active JWTs
- Student verification moderation: approve/reject/revoke
- Category & product type CRUD with relational integrity protection
- Global transaction ledger

6.5. System Behavior & Business Logic

Standard User

1. Listing Submission — Required Fields (enforced via `CreateProductDto`)

- `title`: required, max 120 characters
- `description`: required, non-empty
- `price`: required, minimum 1 ETB
- `category_id` / `product_type_id`: required valid IDs
- `images`: optional array of valid URLs
- `deliverable_file_url`: optional URL

2. Listing Status Transitions

- Creation → `pending`

- Seller edit → resets to `pending`
- Admin approve/reject → `approved` / `rejected` (reject requires reason)
- Hide toggle → independent of status, does not affect moderation state

3. Verified Student Logic

- Purely manual admin review — no automated email-domain check or OCR validation exists in code.
- User uploads `student_id` + `student_id_card_url` → status moves `unverified` → `pending` → admin reviews and sets `verified`/`rejected`.

4. Self-Purchase Prevention

- Enforced server-side in `TransactionsService.create`:

```
if (product.user_id === buyerId) {
  throw new BadRequestException('You cannot purchase your
}
```

5. Purchase Flow (current mock-payment implementation)

1. Buyer clicks "Buy Now" → frontend sends `POST /transactions` with `product_id` + JWT
2. Backend validates token, product existence, and seller ≠ buyer
3. Backend creates a `Transaction` row synchronously: `payment_id: null`, `status: 'completed'`, `is_test: true`
4. Buyer clicks "Download File" → `GET /products/:id/download`
5. Backend verifies a completed transaction exists for that buyer, then returns `deliverable_file_url`

This flow is deliberately built against a mock trigger rather than a card-based processor (see Section 2), validating the full transaction → status → file-unlock pipeline ahead of integrating a locally-relevant payment method.

6. Information Access Before vs After Purchase

- Before: title, description, price, category, type, seller info, images visible. `deliverable_file_url` is explicitly stripped from `findAll`/`findOne` responses.
- After: response includes `has_purchased: true` + `purchased_at`; buyer may call the download endpoint and submit a review.

Admin

1. Listing Review Criteria — no automated rubric or scanner exists; approval/rejection is manual admin judgment.

2. Seller Rejection Notification — a `ProductReview` row is stored with reason; `UsersService.getNotifications` synthesizes a notification from `ProductReview` records for the seller's own products in the last 7 days.

3. User Banning — `AdminService.banUser` sets `is_banned`, `banned_reason`, `banned_at`. Every authenticated request checks `is_banned` in `JwtStrategy.validate` and rejects with 403 if true — banning blocks future logins **and** invalidates already-issued tokens immediately.

4. Admin-Ban Safeguard — `AdminService.banUser` explicitly blocks banning any account with `role: admin`.

5. Verification Flow — admin fetches all profiles via `GET /admin/verifications`, then `PATCH /admin/verifications/:userId` with `{ action: 'approve' | 'reject' | 'revoke', reason? }` updates `is_verified` and `verification_status` accordingly.

---

7. Known Limitations / Not Yet Implemented

1. **Real-World Payment Integration**: Payment processing currently

runs on an internal mock workflow by design (see Section 2). The next step is integrating **Telebirr** (Ethio Telecom), which reflects actual local payment behavior rather than a card-based processor like Stripe. The transaction/status/file-delivery pipeline is already built and tested against this trigger point.
2. **Automated Testing**: Jest is configured; unit/e2e suites are unwritten.
3. **CI/CD Pipelines**: No GitHub Actions or automated deployment pipeline configured yet.
4. **Production Build Optimization**: Frontend and backend currently run via `next dev` / NestJS dev server rather than optimized production builds.
5. **Deferred Features**: Real-time messaging between users, and product review comments from non-buyers, are explicitly out of scope for this phase.

---

8. Environment Setup Instructions

Prerequisites: Node.js v20+, npm v10+, Docker Desktop (optional, for local Postgres)

1. Backend `.env` (`backend/.env`):

```
DATABASE_URL="postgresql://postgres:postgres@localhost:54
JWT_SECRET="your_jwt_access_secret_key"
JWT_REFRESH_SECRET="your_jwt_refresh_secret_key"
JWT_ACCESS_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="7d"
PORT=3002
CLOUDINARY_CLOUD_NAME="your_cloudinary_cloud_name"
CLOUDINARY_API_KEY="your_cloudinary_api_key"
CLOUDINARY_API_SECRET="your_cloudinary_api_secret"
```

2. Local Postgres (optional, dev only): `docker-compose up -d db`

3. Backend:

```
cd backend
npm install
npx prisma db push
npm run start:dev
```

Runs on `http://localhost:3002`

4. Frontend:

```
npm install
npm run dev
```

Runs on `http://localhost:3000`

---

9. Deployment Info

- **Database**: Managed PostgreSQL on **Supabase Cloud** — chosen specifically because it is reachable from both the Vercel-hosted frontend and Railway-hosted backend, which a local database instance cannot be.
- **Media Storage**: Cloudinary Cloud CDN
- **Local dev ports**: Frontend `:3000`, Backend `:3002`

Next steps for full production hardening:

1. Build frontend production assets (`npm run build`)
2. Confirm frontend deployment on Vercel and backend on Railway are both running from production builds
3. Configure production domain, SSL, CORS origin restrictions, and production secrets
4. Integrate Telebirr for real payment processing

---

10. Third-Party Services

1. **Cloudinary** — active on the free/sandbox tier for image and deliverable file storage. *Production transition*: upgrade to a paid tier if storage/bandwidth exceeds free quota.
2. **Payment Provider** — currently a mock internal workflow, by design, in place of a card-based processor poorly suited to the target market. *Production transition*: integrate Telebirr's API for real mobile-money payment processing in Ethiopia.